

# FreshLien

*Same-day foreclosure intelligence — before the crowd*

Master SaaS Product & Technical Document  
v1.0 — June 2026

|                                    |                               |                                     |                                 |
|------------------------------------|-------------------------------|-------------------------------------|---------------------------------|
| \$800B+<br>Distressed Asset Market | \$99/mo<br>Target Entry Price | 30–60 Days<br>Competitor Data Delay | Same-Day<br>FreshLien Freshness |
|------------------------------------|-------------------------------|-------------------------------------|---------------------------------|

**Prepared for:** Waqas Dost — Founder, freshlien.com

**Domain:** freshlien.com

**Status:** Go-to-Build MVP Document

**Stack:** Supabase (PostgreSQL) · FastAPI · React · Scrapy · Mapbox

**Focus:** Pre-Foreclosure · Foreclosure · Probate · Mortgage Default

## Table of Contents

---

1. Product Vision & Market Opportunity
2. Data Model — Full Schema (Supabase / PostgreSQL)
3. Data Sources — What, Where, How
4. Scraper Architecture & ETL Pipeline
5. Backend API — FastAPI Specification
6. Frontend Dashboard — Feature Specification
7. AI / RAG Layer — Natural Language Search
8. Authentication, Subscriptions & Billing
9. Pricing Tiers & Revenue Model
10. Competitive Analysis
11. 60-Day MVP Build Roadmap
12. 90-Day Launch Roadmap
13. Infrastructure & DevOps
14. Legal, Compliance & Data Privacy
15. Go-to-Market Strategy
16. Financial Projections (24-Month)
17. Cursor / Claude Code Prompt Templates
18. Environment Variables & Config Reference

## 1

## Product Vision & Market Opportunity

Why FreshLien wins, who it serves, and the size of the prize

### What FreshLien Is

FreshLien is a B2B SaaS platform that aggregates, normalises, and delivers same-day distressed real estate intelligence — pre-foreclosure (NOD / Lis Pendens), active foreclosure auction data, probate filings, and mortgage default signals — directly scraped from US county recorders, clerk-of-court portals, and public court databases. The product is sold as a subscription dashboard, a REST API, and bulk data exports to real estate investors, wholesale operators, mortgage servicers, attorneys, and hedge funds.

### Core Insight — Speed Is the Product

#### THE ENTIRE PITCH

PropStream (1M+ users, \$99/mo) pulls data through third-party aggregators → 30–60 day lag.

FreshLien scrapes county sources DIRECTLY → same-day or next-day freshness.

Investors who see a pre-foreclosure lead 30 days before competitors can contact the homeowner before anyone else — that first-mover advantage is worth \$10,000s per deal.

### Data Categories FreshLien Covers

| Category           | Filing Type                  | Stage          | Lead Value                        |
|--------------------|------------------------------|----------------|-----------------------------------|
| Pre-Foreclosure    | NOD / Lis Pendens            | Before auction | HIGHEST — homeowner still in home |
| Active Foreclosure | NTS / Sheriff Sale / Auction | Scheduled sale | HIGH — time-sensitive             |
| REO / Bank-Owned   | Post-auction deed            | Lender owns it | MEDIUM — negotiable               |
| Probate            | Probate court filing         | Estate selling | HIGH — motivated seller           |
| Mortgage Default   | Tax lien / HELOC default     | Payment missed | HIGH — early warning signal       |
| Tax Delinquency    | County tax records           | Pre-lien       | MEDIUM — slow burn lead           |

### Target Customer Segments

| Segment              | Pain Point                    | FreshLien Value           | Willingness to Pay |
|----------------------|-------------------------------|---------------------------|--------------------|
| Fix & Flip Investor  | Stale data, miss best deals   | Same-day NOD alerts       | \$79–199/mo        |
| Wholesale Investor   | Manual research 4–6 hrs/week  | 165+ filters, bulk export | \$199/mo           |
| Mortgage Servicer    | No portfolio distress monitor | API flag on any parcel    | \$999+/mo          |
| Real Estate Attorney | Hours of courthouse search    | Full property timeline    | \$199/mo           |
| Hard Money Lender    | Hidden collateral distress    | Collateral risk score     | \$999/mo           |
| Credit Repair Co.    | Finding pre-foreclosure leads | Targeted mail lists       | \$199/mo           |
| Hedge Fund / PE      | Nationwide deal sourcing      | Full API + custom feed    | \$2k–10k/mo        |

| Segment          | Pain Point                | FreshLien Value       | Willingness to Pay |
|------------------|---------------------------|-----------------------|--------------------|
| Probate Attorney | Finding estate properties | Probate filing alerts | \$199/mo           |

2

# Full Database Schema

Supabase (PostgreSQL) — every table, column, index, and relationship

**DATABASE CHOICE**

MVP: Use Supabase free tier (500 MB, 2 CPU). Production: Supabase Pro (\$25/mo) or self-hosted PostgreSQL on AWS RDS (db.t3.micro = ~\$15/mo). PostGIS extension required.

Core Tables Overview

| Table               | Purpose                                             | Row Estimate (Year 1) |
|---------------------|-----------------------------------------------------|-----------------------|
| properties          | Master parcel/property record — one row per address | ~500,000              |
| foreclosure_filings | Every NOD, Lis Pendens, NTS, auction filing         | ~200,000/yr           |
| probate_filings     | Probate court filings linked to property            | ~50,000/yr            |
| tax_liens           | Delinquent tax records and municipal liens          | ~100,000/yr           |
| mortgage_records    | Deed of trust, mortgage origination, satisfaction   | ~300,000/yr           |
| auction_results     | Courthouse sale outcomes — sold price, buyer        | ~80,000/yr            |
| owners              | Deduplicated owner/person entity table              | ~400,000              |
| owner_property_link | Many-to-many owner↔property                         | ~600,000              |
| users               | SaaS user accounts                                  | <10,000               |
| subscriptions       | Stripe subscription state per user                  | <10,000               |
| saved_searches      | User-defined filter presets                         | <50,000               |
| alerts              | Watch-area alert configurations                     | <30,000               |
| alert_deliveries    | Log of every alert email sent                       | <500,000/yr           |
| api_keys            | API key credentials for B2B tier                    | <1,000                |
| skip_trace_log      | Pay-per-use skip trace requests + responses         | <200,000/yr           |
| scrape_jobs         | Scraper run history, status, error log              | <500,000/yr           |
| counties            | Reference table — all 3,143 US counties             | 3,143                 |

SQL: Core Table Definitions

SQL — Supabase / PostgreSQL

```
-- Enable PostGIS & UUID
CREATE EXTENSION IF NOT EXISTS postgis;
CREATE EXTENSION IF NOT EXISTS "uuid-osspl";

-- COUNTIES reference table
CREATE TABLE counties (
  id SERIAL PRIMARY KEY,
```

```

fips CHAR(5) UNIQUE NOT NULL, -- e.g. '12086' = Miami-Dade
name TEXT NOT NULL,
state CHAR(2) NOT NULL, -- e.g. 'FL'
state_full TEXT,
judicial BOOLEAN DEFAULT FALSE, -- TRUE = Lis Pendens state
scraper_url TEXT, -- county portal base URL
active BOOLEAN DEFAULT TRUE
);

-- PROPERTIES master parcel table
CREATE TABLE properties (
id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
parcel_id TEXT, -- county APN/parcel number
county_fips CHAR(5) REFERENCES counties(fips),
address_full TEXT NOT NULL,
address_street TEXT,
address_city TEXT,
address_state CHAR(2),
address_zip TEXT,
property_type TEXT, -- SFR, MFR, CONDO, LAND, COMM
bedrooms INT,
bathrooms NUMERIC(3,1),
sqft INT,
lot_sqft INT,
year_built INT,
assessed_value NUMERIC(12,2),
last_sale_price NUMERIC(12,2),
last_sale_date DATE,
estimated_value NUMERIC(12,2), -- AVM estimate
estimated_equity NUMERIC(12,2), -- est_value - est_balance
tax_status TEXT, -- CURRENT, DELINQUENT, EXEMPT
tax_annual NUMERIC(10,2),
flood_zone TEXT, -- FEMA zone code e.g. AE, X
zoning TEXT,
geom GEOMETRY(Point, 4326), -- PostGIS lat/lng
latitude NUMERIC(10,7),
longitude NUMERIC(10,7),
created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_properties_zip ON properties(address_zip);
CREATE INDEX idx_properties_county ON properties(county_fips);
CREATE INDEX idx_properties_geom ON properties USING GIST(geom);

-- FORECLOSURE FILINGS (core product data)
CREATE TABLE foreclosure_filings (
id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
property_id UUID REFERENCES properties(id),
county_fips CHAR(5) REFERENCES counties(fips),
filing_type TEXT NOT NULL, -- NOD, LIS_PENDENS, NTS, AUCTION, REO
filing_stage TEXT, -- PRE_FORECLOSURE, ACTIVE, POST
case_number TEXT,
filing_date DATE NOT NULL,
recording_date DATE,
auction_date DATE,
judgment_amount NUMERIC(12,2), -- from sample data: Approx Judgment

```

```

default_amount NUMERIC(12,2),
lender_name TEXT,
lender_type TEXT, -- BANK, CREDIT_UNION, PRIVATE, GOV
trustee_name TEXT,
attorney_name TEXT,
-- Defendants (from sample CSV)
defendants TEXT[], -- array of defendant names
defendant_primary TEXT, -- first named defendant
-- Additional lien holders
junior_liens TEXT[], -- HOA, IRS, STATE, etc.
federal_lien BOOLEAN DEFAULT FALSE, -- USA as defendant = federal lien
state_lien BOOLEAN DEFAULT FALSE,
hoa_lien BOOLEAN DEFAULT FALSE,
-- Computed fields
days_to_auction INT, -- computed from auction_date
days_since_filing INT, -- computed from filing_date
-- Source tracking
source_url TEXT,
source_portal TEXT,
property_id_raw TEXT, -- from sample: Property ID col
scrape_job_id UUID,
created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_ff_property ON foreclosure_filings(property_id);
CREATE INDEX idx_ff_county ON foreclosure_filings(county_fips);
CREATE INDEX idx_ff_filing_date ON foreclosure_filings(filing_date DESC);
CREATE INDEX idx_ff_auction_date ON foreclosure_filings(auction_date);
CREATE INDEX idx_ff_type ON foreclosure_filings(filing_type);

-- PROBATE FILINGS
CREATE TABLE probate_filings (
id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
property_id UUID REFERENCES properties(id),
county_fips CHAR(5) REFERENCES counties(fips),
case_number TEXT,
decendent_name TEXT,
estate_value NUMERIC(12,2),
filing_date DATE,
hearing_date DATE,
attorney TEXT,
executor TEXT,
status TEXT, -- OPEN, CLOSED, CONTESTED
source_url TEXT,
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- TAX LIENS
CREATE TABLE tax_liens (
id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
property_id UUID REFERENCES properties(id),
county_fips CHAR(5),
lien_type TEXT, -- PROPERTY_TAX, MUNICIPAL, IRS
lien_amount NUMERIC(12,2),
years_delinquent INT,
filing_date DATE,

```

```
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- OWNERS (deduplicated entity)
CREATE TABLE owners (
id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
full_name TEXT,
name_variants TEXT[], -- AKAs from court docs
mailing_address TEXT,
-- skip trace appended fields
phone_numbers TEXT[],
emails TEXT[],
skip_traced_at TIMESTAMPTZ,
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- USERS & SUBSCRIPTIONS
CREATE TABLE users (
id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
email TEXT UNIQUE NOT NULL,
name TEXT,
company TEXT,
role TEXT DEFAULT 'investor', -- investor, servicer, attorney, lender
created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE subscriptions (
id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
user_id UUID REFERENCES users(id),
stripe_customer_id TEXT,
stripe_sub_id TEXT,
plan TEXT NOT NULL, -- STARTER, PRO, ENTERPRISE
status TEXT, -- active, trialing, canceled
states_allowed TEXT[], -- e.g. ['FL','TX','CA']
api_calls_limit INT,
views_limit INT,
current_period_start DATE,
current_period_end DATE,
created_at TIMESTAMPTZ DEFAULT NOW()
);
```



## 3

## Data Sources

Every source, URL pattern, filing type, cost, and priority

### Source Priority Matrix

| Source                    | Data Type                     | Cost            | Freshness | Priority          |
|---------------------------|-------------------------------|-----------------|-----------|-------------------|
| County Recorder Portals   | NOD, NTS, Deed of Trust       | Free            | Same-day  | P0 — Build first  |
| Clerk of Court (Judicial) | Lis Pendens, foreclosure case | Free            | Same-day  | P0 — FL/NJ/NY     |
| County Assessor / Tax     | APN, value, owner, tax status | Free            | Weekly    | P0 — enrich all   |
| PACER (Federal Courts)    | Bankruptcy Ch.7 / Ch.13       | \$0.10/pg       | Same-day  | P1 — cross-ref    |
| HUD.gov REO               | Bank-owned listings           | Free            | Weekly    | P1 — REO stage    |
| FHFA (Fannie/Freddie)     | GSE foreclosure data          | Free            | Monthly   | P1 — portfolio    |
| Probate Court Portals     | Estate filings                | Free            | Weekly    | P1 — probate tier |
| ATTOM API (gap fill)      | Any county, all data types    | \$200–500/mo    | Daily     | P2 — rural gaps   |
| FEMA Flood Maps API       | Flood zone per parcel         | Free            | Annual    | P1 — risk layer   |
| Census TIGER / Zoning     | Parcel geometry, zoning       | Free            | Annual    | P1 — geo layer    |
| BatchSkipTracing API      | Phone, email per owner        | \$0.05–0.15/rec | On-demand | P2 — add-on       |

### Sample Data Structure — Burlington County NJ Format

The uploaded sample data shows the exact format scraped from Burlington County, NJ. Every filing row maps to these fields in the foreclosure\_filings and properties tables:

| CSV Column              | DB Table                      | DB Column                 | Notes                                   |
|-------------------------|-------------------------------|---------------------------|-----------------------------------------|
| Property ID             | foreclosure_filings           | property_id_raw           | Court docket / internal ID              |
| Address                 | properties                    | address_full              | Parse into street/city/state/zip        |
| Defendant               | owners + owner_property_links | full_name / name_variants | Split on semicolons; flag AKAs          |
| Sales Date              | foreclosure_filings           | auction_date              | The scheduled sheriff/court sale        |
| Approx Judgment         | foreclosure_filings           | judgment_amount           | Parse currency → NUMERIC                |
| (derived) Snapshot Date | scrape_jobs                   | run_date                  | Header row date from source             |
| (derived) State         | properties                    | address_state             | Parse from address string               |
| (derived) County        | properties / ff               | county_fips               | Lookup from county name                 |
| (derived) Federal Lien  | foreclosure_filings           | federal_lien              | TRUE if 'UNITED STATES OF AMERICA' in c |
| (derived) State Lien    | foreclosure_filings           | state_lien                | TRUE if 'STATE OF NEW JERSEY' etc.      |
| (derived) HOA Lien      | foreclosure_filings           | hoa_lien                  | TRUE if 'ASSOCIATION' in defendants     |

**SAMPLE DATA NOTES**

The NJ Burlington County sample has 19 filings across two auction dates (11/20/2025 & 12/4/2025).

Judgment amounts range from \$35K to \$494K. Multiple defendants per filing (semicolon-delimited).

Federal liens (USA), state liens (NJ), and HOA liens appear in the defendants column — always parse these.

## 4

## Scraper Architecture & ETL Pipeline

Scrapy + Selenium → S3 → normalise → Supabase — full spec

### Architecture Overview

Each county gets its own Scrapy spider. Spiders are scheduled by Apache Airflow (or a lightweight cron on EC2) to run at 6:00 AM EST daily. Raw HTML/JSON is archived to S3. The ETL normaliser reads from S3, parses fields, geocodes addresses, detects lien types, and upserts into Supabase via the PostgREST API.

### File Structure

#### PROJECT STRUCTURE

```

freshlien-scraper/
├── spiders/
│   ├── base_spider.py # shared auth, retry, CAPTCHA logic
│   ├── fl_miami_dade.py # Florida – Miami-Dade Clerk of Court
│   ├── fl_broward.py
│   ├── fl_palm_beach.py
│   ├── tx_harris.py # Texas – Harris County (Houston)
│   ├── tx_dallas.py
│   ├── ca_la.py # California – LA County Recorder
│   ├── nj_burlington.py # NJ Burlington (matches sample data)
│   └── ... (30 counties MVP)
├── etl/
│   ├── normaliser.py # field parsing, address splitting
│   ├── geocoder.py # Google Maps or Nominatim
│   ├── lien_detector.py # parse defendants → lien flags
│   ├── upsert.py # Supabase PostgREST upsert
│   ├── assessor_enricher.py # merge county assessor data
│   └── dags/
│       ├── daily_scrape.py # Airflow DAG
│       └── settings.py # Scrapy settings
└── requirements.txt

```

### NJ Burlington Scraper — Concrete Example

#### spiders/nj\_burlington.py

```

import scrapy, re
from .base_spider import BaseForeclosureSpider
from etl.normaliser import normalise_filing
from etl.lien_detector import detect_liens

class NJBurlingtonSpider(BaseForeclosureSpider):
    name = 'nj_burlington'
    county_fips = '34005'
    base_url = 'https://www.burlingtoncountynj.gov/sheriff/foreclosure'

    def parse(self, response):
        for row in response.css('table.foreclosure-list tr:not(:first-child)'):
            cols = row.css('td::text').getall()
            if len(cols) < 5: continue
            raw = {
                'property_id_raw': cols[0].strip(),
                'address_full': cols[1].strip(),

```

```
'defendants_raw': cols[2].strip(),
'auction_date': cols[3].strip(),
'judgment_amount': cols[4].strip(),
'county_fips': self.county_fips,
'filing_type': 'LIS_PENDENS',
'source_url': response.url,
}
filing = normalise_filing(raw)
filing.update(detect_liens(raw['defendants_raw']))
yield filing
```

## Lien Detector Logic

[etl/lien\\_detector.py](#)

```
import re

FEDERAL_KEYWORDS = ['UNITED STATES OF AMERICA', 'U.S.A.', 'IRS',
'SMALL BUSINESS ADMINISTRATION']
STATE_KEYWORDS = ['STATE OF NEW JERSEY', 'STATE OF FLORIDA',
'NJHMFA', 'NEW JERSEY HOUSING AND MORTGAGE']
HOA_KEYWORDS = ['ASSOCIATION', 'HOA', 'CONDOMINIUM', 'COMMUNITY']
PROBATE_KEYWORDS = ['ESTATE OF', 'EXECUTRIX', 'ADMINISTRATOR OF']

def detect_liens(defendants_raw: str) -> dict:
    d = defendants_raw.upper()
    names = [n.strip() for n in defendants_raw.split(';')]
    return {
        'defendants': names,
        'defendant_primary': names[0] if names else None,
        'junior_liens': [n for n in names if any(k in n.upper()
for k in HOA_KEYWORDS+STATE_KEYWORDS+FEDERAL_KEYWORDS)],
        'federal_lien': any(k in d for k in FEDERAL_KEYWORDS),
        'state_lien': any(k in d for k in STATE_KEYWORDS),
        'hoa_lien': any(k in d for k in HOA_KEYWORDS),
        'probate_flag': any(k in d for k in PROBATE_KEYWORDS),
    }
```

## 5

## Backend API — FastAPI Specification

Every endpoint, request/response schema, and auth rule

### Technology Stack

FastAPI (Python 3.11) · Supabase (PostgreSQL + PostGIS) · Redis (hot cache) · Stripe (billing) · SendGrid (email alerts) · Resend (transactional) · JWT (supabase-py auth) · Railway or Render for free-tier hosting initially.

### API Endpoints

| Method | Endpoint                     | Auth       | Description                          |
|--------|------------------------------|------------|--------------------------------------|
| GET    | /api/v1/foreclosures         | JWT        | List/search filings with 50+ filters |
| GET    | /api/v1/foreclosures/{id}    | JWT        | Full property detail with timeline   |
| GET    | /api/v1/properties/{id}      | JWT        | Property record + all linked filings |
| GET    | /api/v1/search               | JWT        | Full-text + geo search               |
| GET    | /api/v1/map/clusters         | JWT        | GeoJSON clusters for map view        |
| GET    | /api/v1/stats/county/{fips}  | JWT        | County-level filing stats            |
| POST   | /api/v1/alerts               | JWT        | Create watch-area alert              |
| GET    | /api/v1/alerts               | JWT        | List user's alerts                   |
| DELETE | /api/v1/alerts/{id}          | JWT        | Delete alert                         |
| POST   | /api/v1/export/csv           | JWT        | Export filtered results as CSV       |
| POST   | /api/v1/skip-trace           | JWT+PRO    | Append phone/email to owner          |
| GET    | /api/v1/risk-score/{prop_id} | JWT        | Collateral risk score (0–100)        |
| GET    | /api/v1/portfolio/monitor    | JWT+ENT    | Portfolio distress check             |
| POST   | /api/v1/api-keys             | JWT+ENT    | Generate B2B API key                 |
| GET    | /api/v1/usage                | JWT        | API call usage this billing period   |
| POST   | /api/v1/webhooks/stripe      | Stripe-sig | Stripe webhook handler               |

### Key Query Parameters — /api/v1/foreclosures

| Parameter    | Type  | Example         | Description         |
|--------------|-------|-----------------|---------------------|
| state        | str[] | FL,TX,CA        | Filter by state(s)  |
| county_fips  | str[] | 12086,48201     | Filter by FIPS code |
| zip          | str[] | 33101,77002     | Filter by ZIP code  |
| filing_type  | enum  | NOD,LIS_PENDENS | Filing category     |
| filing_stage | enum  | PRE_FORECLOSURE | Stage of process    |

| Parameter             | Type  | Example          | Description                                |
|-----------------------|-------|------------------|--------------------------------------------|
| days_since_filing_max | int   | 30               | Filed within N days                        |
| auction_date_from     | date  | 2025-11-01       | Auction date range start                   |
| auction_date_to       | date  | 2025-12-31       | Auction date range end                     |
| judgment_min          | float | 50000            | Min judgment amount                        |
| judgment_max          | float | 500000           | Max judgment amount                        |
| equity_min_pct        | float | 20               | Min estimated equity %                     |
| federal_lien          | bool  | false            | Exclude/include federal liens              |
| property_type         | enum  | SFR,CONDO        | Property type filter                       |
| has_auction_date      | bool  | true             | Only filings with scheduled sale           |
| probate               | bool  | true             | Include probate-flagged filings            |
| sort                  | str   | filing_date:desc | Sort field and direction                   |
| limit                 | int   | 50               | Page size (max 200 Starter, unlimited Pro) |
| offset                | int   | 0                | Pagination offset                          |

## FastAPI Code — Main Application

### main.py — FastAPI

```
# main.py
from fastapi import FastAPI, Depends, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from supabase import create_client
import redis, os

app = FastAPI(title='FreshLien API', version='1.0')
app.add_middleware(CORSMiddleware, allow_origins=['*'],
allow_methods=['*'], allow_headers=['*'])

supabase = create_client(os.environ['SUPABASE_URL'],
os.environ['SUPABASE_SERVICE_KEY'])
cache = redis.Redis.from_url(os.environ['REDIS_URL'])

@app.get('/api/v1/foreclosures')
async def list_foreclosures(
state: str = None, county_fips: str = None,
zip: str = None, filing_type: str = None,
judgment_min: float = None, judgment_max: float = None,
days_since_filing_max: int = None,
federal_lien: bool = None,
sort: str = 'filing_date:desc',
limit: int = 50, offset: int = 0,
user = Depends(get_current_user)
):
    cache_key = f'ff:{state}:{zip}:{filing_type}:{limit}:{offset}'
    cached = cache.get(cache_key)
    if cached: return json.loads(cached)

    q = supabase.table('foreclosure_filings_view').select('*')
    if state: q = q.in_('address_state', state.split(','))
    if county_fips: q = q.in_('county_fips', county_fips.split(','))
    if zip: q = q.in_('address_zip', zip.split(','))
    if filing_type: q = q.in_('filing_type', filing_type.split(','))
    if judgment_min: q = q.gte('judgment_amount', judgment_min)
    if judgment_max: q = q.lte('judgment_amount', judgment_max)
    if federal_lien is not None: q = q.eq('federal_lien', federal_lien)
    if days_since_filing_max:
        from datetime import date, timedelta
        cutoff = date.today() - timedelta(days=days_since_filing_max)
        q = q.gte('filing_date', str(cutoff))
    sort_col, sort_dir = (sort+':desc').split(':')[2]
    q = q.order(sort_col, desc=(sort_dir=='desc'))
    q = q.range(offset, offset + limit - 1)
    result = q.execute()
    cache.setex(cache_key, 300, json.dumps(result.data))
    return result.data
```



# Frontend Dashboard

React + Mapbox + Tailwind — every screen, component, and interaction

## Tech Stack

React 18 · Vite · Tailwind CSS · shadcn/ui components · Mapbox GL JS · React Query (TanStack) · Zustand (state) · Recharts (analytics) · Supabase Auth UI. Deploy: Vercel (free tier for MVP).

## Application Routes

| Route                   | Page / Component                         | Access Level |
|-------------------------|------------------------------------------|--------------|
| /                       | Landing page (marketing)                 | Public       |
| /login                  | Auth — email/password + Google OAuth     | Public       |
| /dashboard              | Map view + filter sidebar (main product) | Starter+     |
| /dashboard/list         | Table/list view of filings               | Starter+     |
| /dashboard/property/:id | Property detail drawer/page              | Starter+     |
| /dashboard/alerts       | Manage watch-area alerts                 | Starter+     |
| /dashboard/saved        | Saved searches                           | Starter+     |
| /dashboard/exports      | Export history + download                | Starter+     |
| /dashboard/skip-trace   | Skip trace credits & history             | Pro+         |
| /dashboard/api          | API key management                       | Enterprise   |
| /dashboard/portfolio    | Portfolio monitor                        | Enterprise   |
| /analytics              | Market analytics & charts                | Pro+         |
| /settings               | Account + billing + subscription         | All          |
| /admin                  | Internal data ops dashboard              | Admin only   |

## Dashboard Map View — Component Spec

The map view is the core product screen. It has three zones:

| Zone                 | Component              | Description                                                                        |
|----------------------|------------------------|------------------------------------------------------------------------------------|
| Left sidebar (320px) | FilterPanel            | All search filters: state, county, ZIP, type, date, judgment range, equity, lien t |
| Center (flex)        | MapView (Mapbox GL)    | Clustered pins for all matching filings. Click pin → PropertyDrawer slides in f    |
| Right drawer (420px) | PropertyDrawer         | Full detail: address, owner, judgment, auction date, lien holders, equity estim    |
| Top bar              | SearchBar + ViewToggle | Address search, map/list toggle, active filter tags, result count, export CSV b    |
| Bottom bar           | ResultStats            | Total results, avg judgment, auction calendar strip                                |



## Map Pin Color Coding

| Color  | Hex     | Meaning                               |
|--------|---------|---------------------------------------|
| Red    | #E63946 | Auction in < 7 days — URGENT          |
| Orange | #F4A261 | Auction in 7–30 days                  |
| Yellow | #FFD166 | Auction in 30–90 days                 |
| Blue   | #00B4D8 | Pre-foreclosure — no auction date yet |
| Purple | #7B2D8B | Probate filing                        |
| Gray   | #94A3B8 | REO / post-auction                    |

7

# AI / RAG Layer

Natural language search + property risk reports using Claude / GPT-4o

Features

| Feature                    | Description                                                            | Model                      | Tier          |
|----------------------------|------------------------------------------------------------------------|----------------------------|---------------|
| Natural Language Search    | "Show 3-bed pre-foreclosures in Miami with 50%+ equity, no flood risk" | Claude 3.5 Sonnet / GPT-4o | Pro+          |
| Property Risk Report       | Plain-English explanation of all liens, risks, auction timeline        | GPT-4o                     | \$4.99/report |
| Market Summary             | "What's happening in ZIP 33101 this week?" — @AndreMalik               | Claude 3.5 Sonnet          | Pro+          |
| Document Parser            | Upload a lien document PDF → extract fields automatically              | Claude 3.5 Sonnet          | Enterprise    |
| Distress Score Explanation | Explain why a property scored 87/100 in plain English                  | GPT-4o                     | Pro+          |

NL Search Prompt Template

AI PROMPT — NL SEARCH PARSER

```
SYSTEM: You are FreshLien's query parser. Convert natural language queries
into a structured JSON filter object. Valid fields: state (2-char array),
county_fips (array), zip (array), filing_type (NOD|LIS_PENDENS|NTS|AUCTION|REO),
property_type (SFR|CONDO|MFR|LAND), bedrooms_min, equity_min_pct,
judgment_min, judgment_max, federal_lien (bool), flood_zone_exclude (bool),
days_since_filing_max, has_auction_date (bool).
Respond ONLY with valid JSON, no explanation.

USER: Show me 3-bed pre-foreclosures in Miami with 50%+ equity and no flood risk

RESPONSE:
{
  "state": ["FL"],
  "county_fips": ["12086"],
  "filing_type": "LIS_PENDENS",
  "bedrooms_min": 3,
  "equity_min_pct": 50,
  "flood_zone_exclude": true
}
```

## 8

## Authentication, Subscriptions & Billing

Supabase Auth + Stripe — complete implementation notes

### Auth Flow

Use Supabase Auth (built-in). Email/password + Google OAuth. On sign-up, create a users row and a subscriptions row with plan=TRIAL (7-day free). JWT is issued by Supabase and validated on every FastAPI request via supabase-py.

### Stripe Integration

- Create 3 Stripe Products: Starter (\$79/mo), Pro (\$199/mo), Enterprise (\$999/mo)
- Create Stripe Checkout Session on plan selection → redirect to Stripe hosted page
- Webhook: stripe.checkout.session.completed → set subscriptions.status='active', plan=X
- Webhook: customer.subscription.deleted → set status='canceled', block API access
- Add-on: skip\_trace charges → Stripe metered billing or one-time PaymentIntent (\$0.25/rec)
- Show current usage vs limits in /settings page (views used, API calls, skip traces)

### Rate Limiting by Plan

| Feature             | Starter (\$79) | Pro (\$199)    | Enterprise (\$999) |
|---------------------|----------------|----------------|--------------------|
| States              | 1              | 5              | All 50             |
| Property views/mo   | 500            | Unlimited      | Unlimited          |
| CSV export rows     | 200/export     | 5,000/export   | Unlimited          |
| Alert watch areas   | 1 ZIP          | 10 ZIPs        | Unlimited polygons |
| API calls/mo        | None           | 1,000          | Unlimited          |
| Skip trace          | Not available  | \$0.25/rec     | \$0.20/rec (bulk)  |
| AI property reports | Not available  | \$4.99/report  | \$2.99/report      |
| Portfolio monitor   | Not available  | Not available  | Included           |
| SLA / support       | Email          | Priority email | Dedicated Slack    |



# Pricing Model & Revenue Projections

3-tier SaaS + high-margin add-ons — 24-month model

## Revenue Projections (Conservative)

| Revenue Line                | Month 3    | Month 6      | Month 12       | Month 24       |
|-----------------------------|------------|--------------|----------------|----------------|
| Starter @\$79 (users)       | \$790 (10) | \$3,950 (50) | \$11,850 (150) | \$31,600 (400) |
| Pro @\$199 (users)          | \$995 (5)  | \$5,970 (30) | \$19,900 (100) | \$59,700 (300) |
| Enterprise @\$999 (clients) | \$0        | \$1,998 (2)  | \$9,990 (10)   | \$29,970 (30)  |
| Skip Trace add-on           | \$200      | \$800        | \$3,000        | \$9,000        |
| AI Reports add-on           | \$0        | \$500        | \$2,000        | \$6,000        |
| Direct mail lists           | \$0        | \$300        | \$1,000        | \$3,000        |
| TOTAL MRR                   | ~\$2k      | ~\$14k       | ~\$48k         | ~\$139k        |
| TOTAL ARR                   | ~\$24k     | ~\$168k      | ~\$576k        | ~\$1.67M       |

## Monthly Operating Costs

| Expense               | Month 1–3 | Month 6+ | Notes                         |
|-----------------------|-----------|----------|-------------------------------|
| Supabase              | \$0       | \$25     | Free tier MVP → Pro at scale  |
| AWS EC2 (scrapers)    | \$30      | \$120    | t3.medium for 30 spiders      |
| AWS S3 (raw archive)  | \$5       | \$20     | Cheap storage                 |
| Redis (Upstash free)  | \$0       | \$20     | Free tier → paid at scale     |
| ATTOM API gap fill    | \$200     | \$200    | Reduce as coverage grows      |
| Google Maps Geocoding | \$0       | \$50     | Use Nominatim free first      |
| LLM API (Claude/GPT)  | \$20      | \$100    | AI features only              |
| Stripe fees           | \$60      | \$420    | 2.9% + \$0.30 per transaction |
| SendGrid / Resend     | \$0       | \$20     | Free tier covers launch       |
| Domain + tools        | \$15      | \$15     | freshlien.com + Cloudflare    |
| TOTAL COSTS           | ~\$330    | ~\$970   | ~92% net margin at Month 6    |

10

## Competitive Analysis

How FreshLien beats every competitor on the only metric that matters: freshness

| Competitor    | Price        | Data Lag   | Probate | API     | FreshLien Edge           |
|---------------|--------------|------------|---------|---------|--------------------------|
| PropStream    | \$99/mo      | 30–60 days | No      | Limited | Same-day + probate + API |
| BatchData     | \$500/mo     | 7–14 days  | No      | Yes     | 10x cheaper, has UI      |
| ATTOM Data    | \$5k–50k/yr  | 3–7 days   | Partial | Yes     | 20–100x cheaper          |
| PropertyShark | \$150–275/mo | 3–7 days   | No      | No      | API + probate + cheaper  |
| CoreLogic     | \$10k+/yr    | 1–3 days   | No      | Yes     | Affordable for SMB       |
| REDX          | \$70–150/mo  | 7–30 days  | No      | No      | Fresher + richer data    |
| FreshLien     | \$79–999/mo  | SAME-DAY   | YES     | YES     | — THE BENCHMARK —        |

### CORE MOAT

The entire competitive moat = county-direct scraping → same-day data → investor contacts homeowner BEFORE PropStream users even see the filing. That's the pitch. Never dilute it.

11

## 60-Day MVP Build Scope

Smallest shippable product that investors will pay for

| Feature                                         | In MVP? | Notes                       |
|-------------------------------------------------|---------|-----------------------------|
| 3 states (FL, TX, CA) + NJ Burlington (sample)  | YES     | 30 county scrapers          |
| Supabase DB (free tier)                         | YES     | No PostGIS needed for MVP   |
| FastAPI backend on Render.com (free)            | YES     | Deploy in 30 min            |
| React dashboard with Mapbox map                 | YES     | Vercel free deploy          |
| Filter panel (state, ZIP, type, date, judgment) | YES     | Core product                |
| Property detail drawer                          | YES     | All scraped fields          |
| CSV export (200 rows — Starter limit)           | YES     | Papa Parse                  |
| Email alerts (1 ZIP — Starter)                  | YES     | SendGrid cron               |
| Stripe subscriptions (Starter + Pro)            | YES     | Launch day billing          |
| Skip trace add-on                               | YES     | BatchSkipTracing API        |
| AI natural language search                      | NO      | v2 feature                  |
| Portfolio monitoring                            | NO      | Enterprise v2               |
| All 50 states                                   | NO      | Add 5 states/sprint         |
| Probate court scraping                          | NO      | Planned v1.5                |
| Mobile app                                      | NO      | Not needed — responsive web |

12

## 90-Day Launch Roadmap

Week-by-week build, validate, and launch plan

| Week | Goal                | Tasks                                                                             |
|------|---------------------|-----------------------------------------------------------------------------------|
| 1–2  | Validate demand     | Post in 3 RE investor FB groups. Get 20 waitlist signups. Carrd landing page.     |
| 3–5  | Data pipeline — FL  | Scrapers for Miami-Dade, Broward, Palm Beach. ETL → Supabase. 500 rows in DB.     |
| 6–7  | MVP dashboard       | React + Mapbox. Filter sidebar. Property drawer. Vercel deploy.                   |
| 8    | TX + CA scrapers    | Harris (Houston), Dallas. LA, Riverside. Now 3 states live.                       |
| 9    | NJ scraper (sample) | Burlington County spider. Matches your sample data format exactly.                |
| 10   | Auth + Billing      | Supabase Auth. Stripe Starter + Pro plans. Email alerts (SendGrid).               |
| 11   | Beta users          | 10 free beta users from waitlist. 30-min Zoom each. Collect feedback.             |
| 12   | Fix + Launch        | Fix top 3 complaints. Get 3 testimonials. ProductHunt launch. BiggerPockets post. |
| 13+  | Growth sprint       | Add 3 states/mo. Add probate tier. Launch skip trace add-on. API tier.            |

Free-tier MVP → production scale path

| Service        | Used For                    | Free Tier         | Paid Trigger                 |
|----------------|-----------------------------|-------------------|------------------------------|
| Supabase       | PostgreSQL + Auth + Storage | 500 MB, 2 CPU     | >500 MB or >50k rows/hr      |
| Render.com     | FastAPI backend             | 750 hrs/mo free   | >750 hrs or need >256 MB RAM |
| Vercel         | React frontend              | Unlimited deploys | Team features                |
| Upstash Redis  | API response cache          | 10k req/day       | >10k req/day                 |
| SendGrid       | Alert emails                | 100 emails/day    | >100/day or custom domain    |
| GitHub Actions | CI/CD                       | 2000 min/mo       | Rarely exceeded              |
| Cloudflare     | CDN + SSL + DNS             | Free forever      | Only for advanced features   |

## freshlien.com · Confidential · Page 24



## 14

## Legal, Compliance & Data Privacy

Public record = legal to scrape, but know the rules

- All foreclosure, probate, and court filings are PUBLIC RECORD under US law. You have the right to access and republish them.
- The FCRA (Fair Credit Reporting Act) applies if you sell data for credit, insurance, or employment decisions. FreshLien sells for REAL ESTATE investment only — not FCRA-regulated.
- GLBA (Gramm-Leach-Bliley) does NOT apply — you are not a financial institution.
- Skip trace data: obtain ONLY from FCRA-compliant providers (TLO, IDI, BatchSkipTracing all certify this). Your TOS must require users to use skip trace data lawfully.
- GDPR / international: Your customers are US-based. Your servers should be in the US. Minimal GDPR exposure for MVP.
- CAN-SPAM: All alert and marketing emails must include an unsubscribe link. Use SendGrid's built-in compliance tools.
- Terms of Service for each county portal: Most prohibit commercial resale. Mitigate by: (a) only displaying derived data, not raw portal HTML; (b) staying below aggressive request rates (1 req/sec max); (c) respecting robots.txt.
- Incorporate as LLC in Wyoming or Delaware (~\$100/yr). Opens US bank account. Use Mercury Bank (free, online, Pakistan-friendly).
- Legal pages required at launch: Privacy Policy, Terms of Service, Data Use Policy. Use Termly.io (\$10/mo) to auto-generate.

15

# Go-to-Market Strategy

How to get your first 100 paying customers from Pakistan

| Channel                      | Action                                                      | Expected Result                                | Cost          |
|------------------------------|-------------------------------------------------------------|------------------------------------------------|---------------|
| BiggerPockets Forum          | Weekly posts sharing county data insights                   | 100+300 visitors/post                          | Free          |
| Facebook Groups              | "Real Estate Investing" groups (1M+ members)                | 50-200 signups/post, share free data snapshots | Free          |
| Reddit r/realestateinvesting | Share NJ/FL/TX data insights, tools AM                      | 500-2k views/post                              | Free          |
| YouTube SEO                  | "How to find pre-foreclosures in [city] before anyone else" | Long tail organic traffic                      | Time only     |
| Direct outreach              | Email 100 active real estate wholesalers                    | 5-10% response = 5-10 trials                   | Free          |
| ProductHunt                  | Launch day — prep 2 weeks in advance                        | 500-2k visits on launch day                    | \$0           |
| Paid Meta ads                | Target: US, real estate investor interests                  | \$500/wk, 1-20-50 trials                       | \$500/mo      |
| Affiliate program            | Pay \$20/referral to real estate educators                  | Severaged growth                               | Revenue share |

**LAUNCH STRATEGY**

First 30 days: post ONLY in BiggerPockets and Facebook groups. Share actual data (e.g. '47 new Lis Pendens filed in Miami-Dade this week — here are the top 5 by equity'). This IS the marketing. Build credibility with free data → convert 10% to paid. No ads until Month 3.

## 16

## Financial Projections — 24 Month

Conservative, base, and stretch scenarios

| Metric             | Month 1 | Month 3 | Month 6  | Month 12 | Month 18 | Month 24  |
|--------------------|---------|---------|----------|----------|----------|-----------|
| Total paying users | 5       | 25      | 100      | 350      | 650      | 1,100     |
| MRR (blended)      | \$400   | \$2,000 | \$13,800 | \$47,000 | \$88,000 | \$138,000 |
| ARR                | \$4,800 | \$24k   | \$166k   | \$564k   | \$1.06M  | \$1.66M   |
| Operating costs    | \$330   | \$500   | \$970    | \$2,500  | \$4,000  | \$6,000   |
| Net profit/mo      | \$70    | \$1,500 | \$12,800 | \$44,500 | \$84,000 | \$132,000 |
| Net margin         | 17%     | 75%     | 93%      | 95%      | 95%      | 96%       |
| States covered     | 4       | 4       | 10       | 25       | 40       | 50        |
| County scrapers    | 30      | 30      | 80       | 200      | 400      | 600       |

## THE MATH

At Month 12 with 350 users and \$47k MRR, you are earning ~\$44k net/month.

That is \$528k/year net profit from a \$330 upfront investment. This is achievable.

Even at 50% of projections = \$264k/year. Still life-changing from Pakistan.

17

## Cursor / Claude Code Prompt Templates

Copy-paste these into Cursor, Claude Code, or any AI IDE to build each module

### Prompt: Supabase Schema

#### CURSOR / CLAUDE CODE — SUPABASE SCHEMA

```
Create a Supabase PostgreSQL schema for a foreclosure data SaaS called FreshLien.
Include tables: properties, foreclosure_filings, probate_filings, tax_liens, owners,
users, subscriptions, alerts, api_keys, scrape_jobs.
Enable PostGIS extension. Add GiST spatial index on properties.geom.
foreclosure_filings must have: filing_type (NOD/LIS_PENDENS/NTS/AUCTION/REO),
judgment_amount, auction_date, defendants TEXT[], federal_lien BOOL, state_lien BOOL,
hoa_lien BOOL, days_to_auction computed column.
Use UUID primary keys everywhere. Add updated_at trigger on all tables.
```

### Prompt: FastAPI Backend

#### CURSOR / CLAUDE CODE — FASTAPI BACKEND

```
Build a FastAPI application for FreshLien foreclosure data SaaS.
POST /auth endpoints using Supabase Auth (supabase-py).
GET /api/v1/foreclosures with filters: state[], county_fips[], zip[], filing_type,
judgment_min/max, days_since_filing_max, federal_lien bool, sort, limit, offset.
GET /api/v1/foreclosures/{id} returns full property detail joined with owners.
POST /api/v1/alerts creates a watch-area alert (polygon or ZIP).
POST /api/v1/export/csv returns streamed CSV of filtered results.
Add Redis caching (5 min TTL) for list queries.
Add Stripe webhook handler for subscription lifecycle.
Rate-limit by subscription plan (Starter 500 views, Pro unlimited).
```

### Prompt: React Dashboard

#### CURSOR / CLAUDE CODE — REACT DASHBOARD

```
Build a React 18 + Tailwind + shadcn/ui foreclosure intelligence dashboard.
Left sidebar: filter panel with state multiselect, ZIP input, filing_type checkboxes,
judgment range slider ($0-$1M), days_since_filing slider, federal_lien toggle.
Center: Mapbox GL JS map, cluster pins by filing count, color-code by urgency:
red=auction<7days, orange=auction 7-30days, blue=pre-foreclosure no auction date.
Right drawer: slides in on pin click. Shows address, defendant names, judgment amount,
auction date, days until auction, lien holders list (HOA/Federal/State badges),
estimated equity, flood zone badge, skip trace button (Pro only), export row button.
Top bar: address search (geocode), map/list toggle, filter tag chips, result count.
Use React Query for data fetching. Zustand for filter state.
```

### Prompt: Burlington NJ Scraper

#### CURSOR / CLAUDE CODE — BURLINGTON NJ SCRAPER

```
Write a Scrapy spider for Burlington County NJ foreclosure listings.
The source page lists properties with columns: Property ID, Address, Defendant, Sales Date, Approx Judgment.
Defendants are semicolon-delimited. Addresses include alternate mailing addresses.
Parse judgment amounts to float. Parse sales date to ISO date.
Split defendants array. Detect: federal_lien (UNITED STATES OF AMERICA),
```

```
state_lien (STATE OF NEW JERSEY), hoa_lien (ASSOCIATION/CONDOMINIUM in name),  
probate_flag (ESTATE OF/EXECUTRIX in name).  
Upsert to Supabase via PostgREST API. Log run to scrape_jobs table.  
Handle pagination if multiple pages. Retry on 429/503. Archive raw HTML to S3.
```

## Prompt: Email Alert System

### CURSOR / CLAUDE CODE — EMAIL ALERT SYSTEM

```
Build a FastAPI background task that sends email alerts for FreshLien.  
Every 6 hours, for each alert in the alerts table:  
1. Query foreclosure_filings for new filings within the alert's ZIP/polygon  
that were filed since the last alert delivery.  
2. If any new filings found, send email via SendGrid with a table showing:  
address, filing type, judgment amount, auction date, days until auction.  
3. Log delivery to alert_deliveries table.  
Use APScheduler for the background job. Template the email in HTML.  
Include unsubscribe link (CAN-SPAM compliance).
```

## 18

## Config & Quick Reference

All URLs, keys, commands, and links you need on day one

### Key URLs & Services

| Service            | URL / Command             | Purpose                          |
|--------------------|---------------------------|----------------------------------|
| Supabase dashboard | app.supabase.com          | DB, auth, storage management     |
| Vercel deploy      | vercel deploy --prod      | Frontend deploy CLI              |
| Render deploy      | git push → auto-deploy    | Backend FastAPI hosting          |
| Stripe dashboard   | dashboard.stripe.com      | Subscriptions, webhooks, payouts |
| SendGrid           | app.sendgrid.com          | Email alerts setup               |
| Upstash Redis      | console.upstash.com       | Redis cache (free tier)          |
| Mapbox Studio      | studio.mapbox.com         | Map style + access token         |
| Namecheap          | namecheap.com             | freshlien.com domain management  |
| Cloudflare         | cloudflare.com            | DNS + free SSL + CDN             |
| BatchSkipTracing   | batchskiptracing.com      | Skip trace API for owners        |
| ATTOM API          | api.attomdata.com         | County gap-fill data             |
| 2Captcha           | 2captcha.com              | CAPTCHA solving for scrapers     |
| Mercury Bank       | mercury.com               | US business bank account         |
| Wyoming LLC        | wyomingagents.com         | Entity formation ~\$100/yr       |
| ProductHunt        | producthunt.com/posts/new | Launch day listing               |
| BiggerPockets      | biggerpockets.com/forums  | Primary GTM channel              |

### Quick-Start Commands

#### QUICK START COMMANDS

```
# 1. Clone & setup
git clone https://github.com/your-org/freshlien
cd freshlien && cp .env.example .env # fill in your keys

# 2. Backend
cd backend && pip install -r requirements.txt
uvicorn main:app --reload --port 8000

# 3. Frontend
cd frontend && npm install
npm run dev # localhost:5173

# 4. Scraper (one spider)
cd scraper && pip install -r requirements.txt
scrapy crawl nj_burlington -o output/nj_burlington.json
```

```
# 5. Deploy frontend
vercel --prod

# 6. Deploy backend (Render.com)

# Connect GitHub repo → set env vars → auto deploys on push
```

#### NEXT STEPS — HOW TO USE THIS DOCUMENT

You now have the complete master document for FreshLien.

Give this PDF to Cursor, Claude Code, or any AI IDE as context.

Start with Section 17 prompts — copy any prompt directly into your IDE.

Build in order: Schema → Scraper (NJ first) → FastAPI → React dashboard → Stripe → Launch.

***freshlien.com***

Same-day foreclosure intelligence — before the crowd

Prepared for Waqas Dost · Confidential · v1.0 · June 2026